



GLITZY

DOC NO. RL-360-STUDIO-A REV. A RIPOSTE LABORATORIES

A studio for building glitch artwork out of chains of operations, and turning the results into cut-ready stickers for etsy.

CONTENTS

▸ What it is	2
▸ Layers	2
▸ Op families	4
▸ Layout	5
▸ Running the tests	5
▸ Things that will bite you	6
▸ Deploying	8

Live at <https://glitzy.hq.ripostelabs.xyz/> (Authelia-gated), engine in LXC 114.

SEC.01

WHAT IT IS

A project is a **chain**: a straight list of nodes, each an op plus its parameters.

```
source.flow → codec.smear → codec.blockquant → colour.palette → matte.shape → matte.border
```

Everything that flows between nodes is a **Clip** – N frames of RGBA, where a still is just N = 1. That is why there is no split between "image effects" and "video effects": an op written for one works on the other unchanged.

Every node's output is cached under `sha1(op, params, upstream_hash)`. Because the key includes the upstream hash, it identifies the whole chain **prefix**, so:

- ▶ editing the last node of an eight-node chain recomputes exactly one node
- ▶ setting a parameter back to a value you tried five minutes ago costs nothing
- ▶ a sweep of twelve variants costs twelve cheap tails, not twelve whole chains

That cache is the difference between a studio and a cook button, and it is the reason the compute lives on the backend rather than in the browser.

SEC.02

LAYERS

A project is a **stack** of those chains, composited bottom to top. Each layer holds one chain plus how it meets the layers under it: blend mode, opacity, fit, scale, offset, clip-to-below, and how a short layer fills a long one's timeline.

top	Layer 2	source.rings → matte.shape	multiply · 60%
	Layer 1	source.media → codec.smear	screen
base	Base	source.flow → colour.palette → matte	normal

Nothing about the chain model changed. A layer's chain is evaluated by the same

evaluator and lands on the same cache; the stack is a second, much smaller graph on top. The composite is itself a cached node, keyed on every layer's output hash plus its placement:

- ▶ changing a blend mode or opacity re-runs **one composite and no ops at all**
- ▶ a layer you are not editing is a cache hit, so the cost of a stack is the layer you are working in
- ▶ a single ordinary layer is not composited at all – it hands back its chain's own cache entry, so a one-layer project is byte for byte what it was before layers existed

Rules worth knowing:

- ▶ **The canvas is the bottom visible layer's own size.** Adding a decorative layer can never resize the artwork you have already been printing. Upper layers are placed on it by their **fit**, and the stack runs as long as its longest layer.
- ▶ **Compositing is the W3C model, not a lerp.** A Multiply layer over a transparent region comes out as the source colour, not black – the alternative is a bug that only shows up in the die-cut contour, after the sticker is printed.
- ▶ **Erase and Keep inside** ignore their own colour and use their alpha as a stencil on everything below. **Clip to below** does the opposite: it restricts a layer to where the stack is already opaque – a texture poured into a silhouette.
- ▶ **Solo** shows the active layer alone. What is on screen is what Keep and the exports use, so a soloed view keeps that single layer, not the stack.
- ▶ A layer with an empty chain is skipped rather than an error, and a hidden layer is never validated – a half-built or broken layer you have switched off cannot block the render of the ones you can see.

The layer settings are declared with the same schema helpers as op parameters and are served from `/api/ops` alongside them, so **adding a blend mode is a backend-only change** too.

SEC.03

OP FAMILIES

CATEGORY	WHAT IT DOES
source	7 numpy generators (flow, truchet, moire, plasma, rings, cells, feedback), uploaded/fetched media, flat colour, gradients
codec	7 ffedit passes that corrupt the MPEG-2 bitstream itself – smear, tail, sink, storm, freeze, blockquant, bleed
pixel	sort, channel shift, displace, slice shuffle, recompress, bitcrush, noise, scanlines, levels, blur
colour	colourways, dither, hue/saturation, posterize, invert, channel swap
geom	resize, transform, mirror/kaleidoscope, tile, wave, polar
matte	key out, 13 silhouettes, die-cut border, backing, trim
time	drift, length, reverse, echo, slit scan, stutter

Ops are registered with a declared parameter schema, and the studio builds every control from `/api/ops`. **Adding an effect is a backend-only change** – the frontend never learns its name.

SEC.04

LAYOUT

glitzyd/	the engine (Python 3 stdlib + numpy + Pillow)
clip.py	the Clip type; frames on disk as PNGs
store.py	content-addressed cache, sources, projects
graph.py	chain and stack evaluation, cache reuse
layers.py	blend modes, placement, the composite itself
jobs.py	worker queue with progress and cancel
ff.py	ffgac/ffedit bridge: encode → glitch → decode
nputil.py	sampling, blur, bounded distance transform, noise
palettes.py	colourways, shared by generators and colour ops
exporters.py	PNG, sequence, GIF, APNG, and the etsch/1 sheet
server.py	HTTP API and static host
ops/	the op library, one module per family
rack/	ffedit qjs scripts (one per codec op)
web/	the studio – native ES modules, no build step
tests/	test_backend.py (186 checks), uitest.mjs (102 checks)

SEC.05

RUNNING THE TESTS

Both suites run inside LXC 114. Neither mocks anything: the backend tests drive the real ffglitch binaries, and the UI test drives real Chromium.

BASH

```

./deploy.sh                                # from x
pct exec 114 -- sh -c 'cd /opt/glitzzy && python3 tests/test_backend.py'
pct exec 114 -- sh -c 'cp /opt/glitzzy/tests/uitest.mjs /root/uitest/glitzzy.mjs \
&& cd /root/uitest && node glitzzy.mjs http://localhost:8090'
```

The UI test must be copied into `/root/uitest` and run from there. Node resolves `playwright-core` relative to the script, so running it out of `/opt/glitzzy/tests` fails to import before it checks anything.

The backend suite needs no running service – it drives the engine library directly against a temporary data directory – so it is safe to run while the live one is up. The UI suite needs a server; point it at a spare port rather than the live one:

BASH

```
GLITZY_DATA=/tmp/glitzzy-uitest GLITZY_PORT=8099 python3 -m glitzzyd.server &
```

Kill that by **PID**. `pkill -f glitzzyd.server` also matches the shell running the `pkill`, so it kills the wrapper, returns 143, and leaves the engine up – which then looks like the kill silently failed.

SEC.06

THINGS THAT WILL BITE YOU

ffedit 0.10.2

- ▶ `-sp` rejects floating point outright and then writes no output file, which reads like a crash. Every codec parameter is an integer; fractions travel percent-scaled and are divided inside the `qjs` script.
- ▶ MV arrays accept only `null` or `MV(x,y)`. A plain `[x,y]` throws.
- ▶ `frame.qscale.slice` is an array of objects keyed by macroblock index, not a 2D array. A `.length` loop over it silently does nothing and writes a byte-identical file.
- ▶ A script that touches nothing still exits 0 and writes a valid file, so a silent no-op looks exactly like success. The engine sha1-compares in and out and says so.
- ▶ Warning `MVs not available` and `concealing N DC/AC/MV errors` on stacked passes are **benign** – a decoder concealing a smeared stream, and a coarser quantiser spending fewer bits. Never treat them as failure.
- ▶ Parse `ffgac` output with the `full` stderr. It prints stream and duration lines near the top, so a tail-only read reports every video as a 0x0 still.

Print

- ▶ Encode widths pad to a multiple of 16, or the macroblock grid stops meeting the frame edge – and the block edges are the aesthetic.
- ▶ Upscaling for print is integer **nearest-neighbour, rounded up**. Bicubic destroys the block edges; rounding down silently prints below the requested DPI.
- ▶ The silhouette and its die-cut border are baked into the artwork's **alpha**, because etsch traces the contour of the transparency. So the sheet asks for **contour at offset: 0** – adding an offset as well cuts a second line outside the border.

Browser

- ▶ `[hidden]` needs `!important`. Any class that sets `display` outranks the UA sheet's `[hidden]{display:none}`, and `jsdom` reports `el.hidden === true` and passes while a real browser shows the element.
- ▶ Panels in a scrolling flex column need `flex:0 0 auto`, or each one is squashed below its content height and overlaps the panel beneath, stealing its clicks.
- ▶ The viewer is a fixed box the artwork fits into. Sizing the box to its content means every switch between a preview and a full render moves the artwork across the screen.
- ▶ `do_POST` must drain the request body **before** replying, even to reject it. Reply without draining and the next request on that keep-alive connection is parsed inside the leftover bytes, which surfaces as a random 501 and a silently dropped upload.

SEC.07

DEPLOYING

`./deploy.sh [--restart]` copies the tree into LXC 114 via its ZFS subvol and restarts `glitzyd`. The unit is `glitzyd.service`, port **8090**, data in `/var/lib/glitzy`.

The old v1 engine still exists on port 8080. It kept its original unit name, `glitchd.service` – that is not a typo and the rename did not touch it, because renaming a running service that this one falls back to would break the only rollback there is. Rollback is one line in `/etc/caddy/sites/160-glitzy.caddy` plus `systemctl reload caddy` on LXC 104.

SOURCE `studio/README.md` SHA `bbcce3898d80` BUILT `2026-08-14`